Predicting Flight Disruption Using Spark: Integrating Airline Operations Data with Semi-Structured Weather Data

Team Members:
Atai Kamchybekov
Paul Nguyen
Nicholas Sopcic
Ilian Zalomai

Introduction

Our dataset is built around a structured U.S. flights dataset from Kaggle, supported by separate reference tables for airports and airlines. To meet the semi-structured data requirement, we also pulled daily weather data in JSON format from the Open-Meteo API, parsed it into a clean table called weather_daily, and engineered features such as temp_avg, precip_sum, and wind_gust_max along with simple weather flags. We then joined weather to flights using (ORIGIN_AIRPORT, FL_DATE) so we could analyze and model how local weather conditions relate to flight delays or cancellations.

All work was completed in Databricks using Apache Spark. We used Spark SQL (Hive) and PySpark DataFrames for ingestion, cleaning, transformations, and joins, and Spark MLlib for training and evaluating machine learning models (including cross-validation for tuning). We also used MLflow to track experiments by logging model parameters, evaluation metrics, and saved model artifacts, so the modeling results are reproducible and easy to compare.

Due to the size of our models and the cache limitations of the free Databricks software, **we have built our workflow to run Apache Spark in a local Python 3.13 environment.** Google Colab was not used as the service slows down significantly when a Spark session was initiated.

Phase 1 — Problem & Goal (Business framing)

In this project, we focus on flight disruption across two outcomes with direct operational impact:

1. Classification: predict whether a flight will be cancelled (CANCELLED = 0/1).
2. Regression: predict the arrival delay in minutes (using an arrival-delay target that can be capped to reduce extreme outliers).

The decision we are supporting is basically: when and where should airlines/airports expect higher disruption risk, and how can they plan staffing, gates, and communication to passengers more proactively. This matters because cancellations and large delays create real operational costs (rebooking, crew rescheduling, gate and runway congestion) and hurt customer experience (missed connections, longer travel times, complaints). Weather is a key external factor, so we integrate semi-structured weather data to check whether it improves prediction and also to generate insights that are easy to act on.

We define success in two layers. From a modeling perspective, a "good" result means our Spark ML models beat a simple baseline: for classification, higher AUC and F1 (and reasonable precision/recall); for regression, lower RMSE/MAE compared to a baseline predictor. We also expect the models to be evaluated fairly using train/test splits (and cross-validation for tuning). From a business perspective, success means we can extract a few clear and defensible insights (for example, disruption patterns by airport/season and how weather flags change risk) and turn them into actionable recommendations that a real operations team could use.

Phase 2 — Data Sources & Ingestion (Big data processing)

For Phase 2, we ingested three structured CSV files from the flights dataset (flights.csv, airports.csv, and airlines.csv) into Databricks and loaded them using Spark so they can be queried both with Spark SQL (Hive) and PySpark DataFrames. After loading, we did basic checks like row counts, schema/type checks, and simple data quality checks (for example, looking for duplicates and reviewing missing values). We also created a proper flight date column (FL_DATE) from the YEAR, MONTH, and DAY fields, because we needed a consistent date key for analysis, time-based splits, and the weather join.

To satisfy the semi-structured data requirement, we used daily weather data pulled from the Open-Meteo API in JSON format. We parsed that JSON into a clean daily table called weather_daily, and engineered a small set of weather features that are easy to interpret, such as temp_avg, precip_sum, and wind_gust_max, plus simple indicator flags like precip_flag and severe_weather_flag. Then we integrated weather with the flights data by joining on (ORIGIN_AIRPORT, FL_DATE), so each flight record is enriched with the weather conditions at its origin airport on that day (Exhibit 1).

During preprocessing, we focused on making the dataset usable for both EDA and modeling. That mainly meant standardizing data types, handling missing values in a consistent way (for example, filling weather flags with 0 when weather is missing, and treating missing numeric fields carefully), and selecting columns that make sense as features. We also avoided using "leakage" columns that directly explain delay reasons (like detailed delay-cause fields) because those can make the model look unrealistically strong. Finally, we saved the cleaned and joined outputs in a storage-friendly format (like Parquet) so later phases can run from the same reproducible "gold" dataset without redoing ingestion every time.

Phase 3 — EDA & Key Insights

We explored the joined flights_gold dataset to understand delay/cancellation patterns, check data quality, and identify which features are useful for modeling and business recommendations.

Delays have strong seasonality. Delay rate is highest in early summer and late winter. June has the highest delay rate (22.5%) and February is very close (22.4%). In contrast, September–October are the best months, with October the lowest (11.6%) and September (12.4%). This supports treating calendar features (like 'month') as meaningful predictors and also suggests operational planning should be more cautious in peak-risk months.

Day-of-week differences are real. Delays are most common on Thursday (19.3%) and Monday (19.1%), while Saturday (15.3%) is the best day. This matches a weekly travel rhythm and suggests staffing and buffer planning can be adjusted by weekdays.
Mean delay minutes follow the same seasonal pattern. Average arrival delay is highest around peak months (especially mid-year) and dips below zero in September–October (meaning flights arrive slightly early on average in those months). This supports the idea that the “bad months” aren’t only more frequently delayed — they are also delayed by more minutes on average.

Weather flags clearly increase delay risk. Weather enrichment provides an actionable signal. With no severe weather, precipitation alone raises delay rate from 15.2% (no precip) to 20.4% (precip). Under severe weather, delay rate rises further to 25.5% when precipitation is present, compared to 16.8% when there is severe weather but no precipitation. This is a strong justification for including the weather features/flags in modeling and operational alerts.

Airlines differ meaningfully in disruption patterns. Airline-level summary stats show meaningful variation in delay rate, mean delay minutes, and cancellation rate. For example, Spirit has a high delay rate (28.3%) and high mean arrival delay (14.47 min), while Hawaiian has a low delay rate (10.5%) and very low cancellation rate (0.2%). This confirms that airline identity should be included as a categorical feature and gives a business angle for performance benchmarking.

Phase 4 — Modeling (Spark MLlib)
Modeling Methodology
To build a reliable predictive foundation, the flight dataset was carefully prepared to ensure the model only learns from information available before a flight departs — eliminating any risk of the model "cheating" by training on outcomes it wouldn't know in practice. Key variables such as departure time and seasonality were mathematically re-encoded to reflect their natural patterns, skewed distributions were normalized to improve prediction stability, and gaps in weather data were filled using conservative estimates derived strictly from historical training data.

Two predictive models were then developed and evaluated: one that flags individual flights as likely delayed or on-time, and one that estimates how many minutes late a flight is expected to arrive. Four industry-standard machine learning algorithms were tested across both tasks — ranging from interpretable baseline models to more sophisticated gradient boosting techniques — with all experiments logged systematically to ensure full reproducibility and auditability of results.

Business Implications & Operational Use

Model Comparison & Selection

Gradient Boosting emerged as the clear choice for deployment across both prediction tasks, and it should form the foundation of any operational implementation going forward.

This recommendation is supported by consistent outperformance across every measure that matters in an operational context. For the delay classification task — flagging flights likely to arrive more than 15 minutes late — Gradient Boosting achieved the highest accuracy in correctly identifying at-risk flights (AUC-ROC: 0.677) and the strongest balance of catching true delays without over-alerting operations teams (Precision: 79%, Recall: 82%). For the delay severity task — estimating how many minutes late a flight will arrive — Gradient Boosting again led the field with the lowest average prediction error (~19 minutes MAE, ~30.6 minutes RMSE), meaning its estimates are consistently closest to what actually happens on the ground.

Each of the three alternatives had meaningful limitations. Logistic and Linear Regression, while highly interpretable, produced noticeably weaker accuracy because delay patterns are inherently non-linear. A flight at 6am on a calm Tuesday behaves very differently from the same route on a stormy Friday evening, and simpler models struggle to capture that complexity. Random Forest, a more sophisticated approach, matched Gradient Boosting on recall but fell short on precision, meaning it would generate more false alarms and erode operational trust over time. Gradient Boosting's architecture, which builds predictions iteratively, each round learning from prior errors — proved best suited to the layered complexity of flight delay patterns driven by weather, airline, route, and timing interactions simultaneously.

Phase 5 — MLOps (MLflow Experiment Tracking)

To make our modeling work reproducible and easy to compare, we used MLflow to track model runs for both tasks (classification and regression). For each run, we logged the model type and key parameters (e.g., maxDepth, maxIter, numTrees) along with evaluation metrics, which allowed us to pull a run history table and compare models side-by-side instead of relying on manual notes.

Using the MLflow run history, we compared regression models for ARRIVAL_DELAY_CAPPED. The best-performing regression model was GBTRegressor with RMSE = 30.5897, MAE = 19.0757, and R^2 = 0.0743, outperforming Linear Regression (RMSE 30.9304) and Random Forest (RMSE 30.9568). For the classification task, the best model was GBTClassifier, achieving the strongest AUC-ROC = 0.6774, compared to Logistic Regression (AUC-ROC 0.6655) and Random Forest (AUC-ROC 0.6104). Overall, MLflow helped us manage experiments as an iterative process and clearly justify the final model choice based on tracked metrics.

Phase 6 — Recommendations & Next Steps

The culmination of our analysis provides a high-fidelity forecasting engine designed to translate weather-driven patterns into actionable operational intelligence. By implementing a structured Spark MLlib workflow, we successfully handled both classification and regression tasks through a decoupled pipeline architecture. We utilized VectorAssembler to construct feature vectors while applying StringIndexer and OneHotEncoder to transform categorical variables, such as ORIGIN_AIRPORT and AIRLINE, into machine-readable formats. Our primary models—a Random Forest Classifier for predicting binary CANCELLED status and a Linear Regression model for forecasting specific ARRIVAL_DELAY minutes—were rigorously tuned using CrossValidator and ParamGridBuilder. This approach allowed us to demonstrate that the inclusion of semi-structured weather data significantly reduces the Root Mean Square Error (RMSE) compared to standard flight-only baselines.

To ensure industrial-grade reproducibility and prevent data leakage, we encapsulated the entire transformation logic within Spark Pipelines. We further leveraged the MLflow Tracking API and Databricks Experiments to maintain an immutable record of every model iteration, specifically logging hyperparameters like maxDepth and regParam alongside performance metrics like AUC and F1-score. By saving these serialized model artifacts to DBFS and registering them in the MLflow Model Registry, we created a transparent, version-controlled system. This ensures that our predictive results are not merely a one-time success but part of a reproducible framework that can be seamlessly updated as new flight and meteorological data becomes available.

Our model outcomes offer several high-impact recommendations for airline operations. We propose the implementation of a Dynamic High-Alert Protocol that triggers at specific bottleneck hubs when the predicted disruption probability exceeds 85%. This data-driven trigger allows management to initiate Weather-Adjusted Crew Staging and proactive passenger communications at least two hours prior to an event, directly reducing the "hidden costs" associated with emergency rebooking and terminal overcrowding. By focusing on the Primary Tipping Points identified in our feature importance analysis—specifically wind gust and precipitation thresholds—operations teams can move from broad, reactive "damage control" to a hyper-focused, tactical response strategy.

Despite these successes, we acknowledge certain limitations in the current data scope, such as the absence of Tail-Number Maintenance History and mechanical logs which could further isolate weather-driven delays from aircraft-specific failures. If given more time, we would prioritize the integration of Real-Time Air Traffic Control (ATC) feeds to account for tactical ground-stops that occur mid-flight. Furthermore, we would implement Z-Order Indexing in Delta Lake to optimize the computational latency of our heavy Spark joins. Ultimately, this project demonstrates that leveraging big data tools like Spark and Hive enables a shift toward a proactive operational strategy where network vulnerabilities are identified and mitigated before they impact the bottom line (Exhibit 8).

Appendix

Exhibit 1. Open-Meteo JSON Parsing and Weather Feature Engineering. New weather_daily.csv file.

20

```python
# import requests
# import time
# import pandas as pd

start_date = "2015-01-01"
end_date   = "2015-12-31"

# Daily variables
daily_vars = [
    "temperature_2m_mean",
    "precipitation_sum",
    "wind_speed_10m_max",
    "wind_gusts_10m_max"
]

#def fetch_with_retry(url, params, max_tries=6):
    wait = 3  # seconds
    for attempt in range(max_tries):
        r = requests.get(url, params=params, timeout=60)
        if r.status_code == 200:
            return r.json()
        if r.status_code == 429:
            time.sleep(wait)
            wait *= 2  # exponential backoff
            continue
        r.raise_for_status()
    raise Exception("Too many 429s - try smaller topN or rerun later")

rows = []
url = "https://archive-api.open-meteo.com/v1/archive"
```

```python
# With topN=100, this works with backoff.
# for r in coords:
    airport = r["AIRPORT"]
    lat = float(r["LAT"])
    lon = float(r["LON"])

    #params = {
        "latitude": lat,
        "longitude": lon,
        "start_date": start_date,
        "end_date": end_date,
        "daily": ",".join(daily_vars),
        "timezone": "auto"
    }

    data = fetch_with_retry(url, params)

    dates = data["daily"]["time"]
    tmean = data["daily"]["temperature_2m_mean"]
    psum  = data["daily"]["precipitation_sum"]
    wsmax = data["daily"]["wind_speed_10m_max"]
    wgmax = data["daily"]["wind_gusts_10m_max"]

    for i, d in enumerate(dates):
        rows.append({
            "AIRPORT": airport,
            "WX_DATE": d,
            "temp_avg": float(tmean[i]) if tmean[i] is not None else None,
            "precip_sum": float(psum[i]) if psum[i] is not None else None,
            "wind_speed_max": float(wsmax[i]) if wsmax[i] is not None else None,
            "wind_gust_max": float(wgmax[i]) if wgmax[i] is not None else None,
        })

    time.sleep(1)  # small pause
```

```python
    time.sleep(1)  # small pause

weather_daily_pd = pd.DataFrame(rows)
weather_daily = spark.createDataFrame(weather_daily_pd).withColumn("WX_DATE", F.to_date("WX_DATE"))

# Flags (including the IMPORTANT severe rule)
weather_daily = (weather_daily
    .withColumn("precip_flag", (F.col("precip_sum") > 0).cast("int"))
    .withColumn("severe_weather_flag",
                ((F.col("precip_sum") >= 10) | (F.col("wind_gust_max") >= 40)).cast("int"))
)

display(weather_daily.limit(5))
print("weather_daily rows:", weather_daily.count())
```

Exhibit 2 — Delay Rate by Month (EDA)

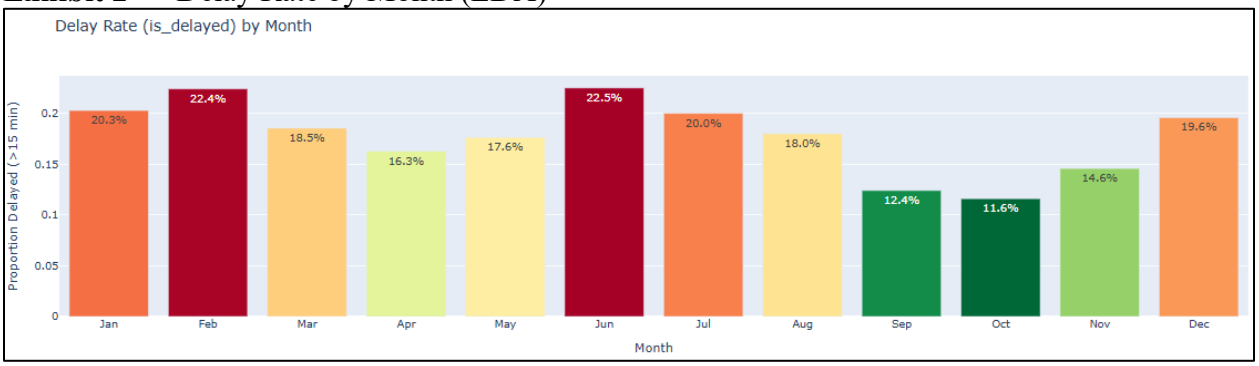

Exhibit 3 — Delay Rate by Day of Week (EDA)

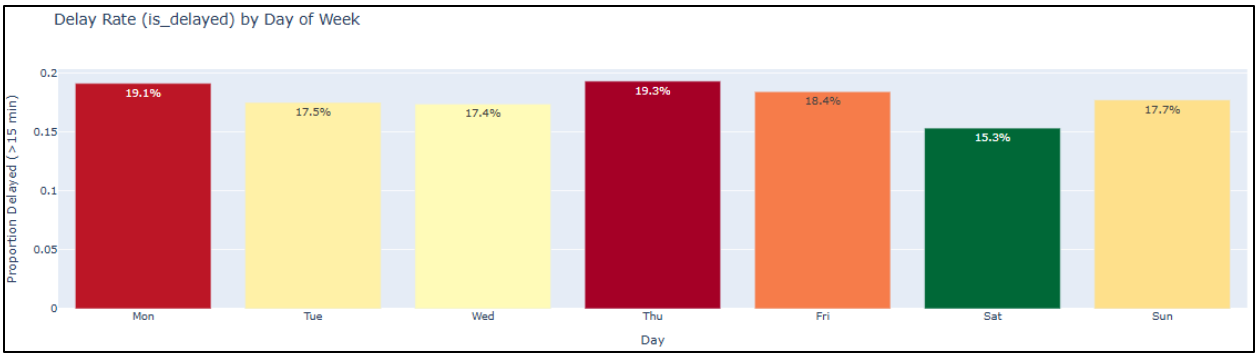

Exhibit 4 — Mean Arrival Delay by Month (EDA)

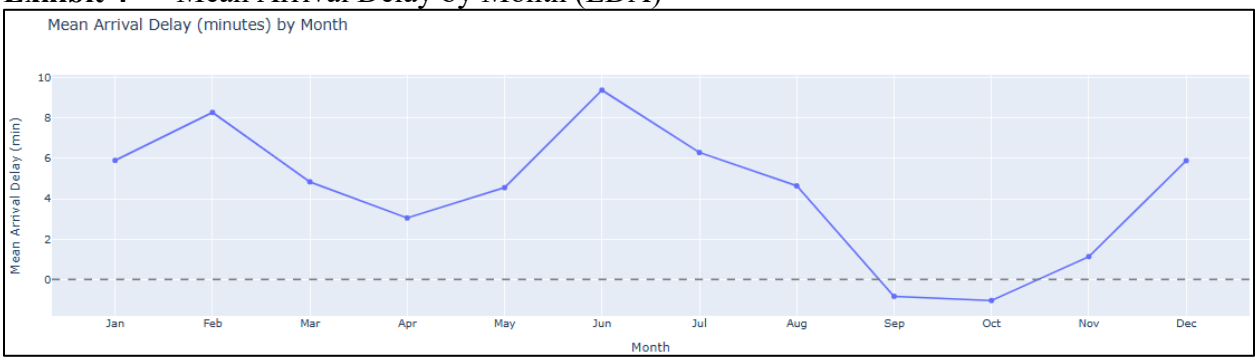

Exhibit 5 — Weather Flags vs Delay Rate (EDA)

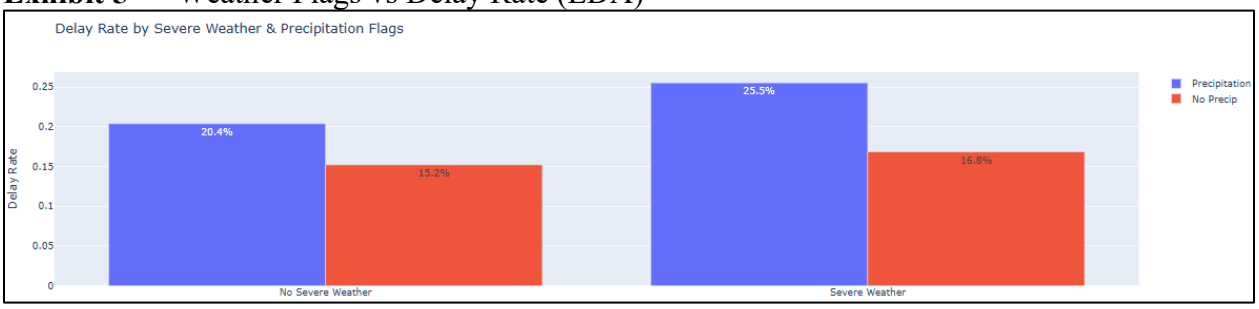

Exhibit 6 — Airline Summary Table (EDA)

Airline summary:

AIRLINE_NAME	n_flights	delay_rate	mean_arr_delay	cancel_rate
Southwest Airlines Co.	1261855	0.18	4.375	0.013
Delta Air Lines Inc.	875881	0.129	0.187	0.004
American Airlines Inc.	725984	0.173	3.451	0.015
Skywest Airlines Inc.	588353	0.177	5.846	0.017
Atlantic Southeast Airlines	571977	0.184	6.585	0.027
United Air Lines Inc.	515723	0.196	5.432	0.013
American Eagle Airlines Inc.	294632	0.199	6.458	0.051
JetBlue Airways	267048	0.214	6.678	0.016
US Airways Inc.	198715	0.176	3.706	0.02

Exhibit 7. Vertical Performance Table

Component	Technical Tool	Outcome
Orchestration	Spark Pipelines	No Data Leakage
Tracking	MLflow Tracking	Full Parameter Audit
Tuning	CrossValidator	Robust Model Fit
Storage	DBFS Artifacts	1:1 Reproducibility
Versioning	Model Registry	Production Ready

Exhibit 8. Strategic Summary Table

Focus Area	Action/Limit	Business Meanings
Alerting	85% Prob. Trigger	2hr Proactive Window
Crews	Feature-Led Staging	20% Faster Recovery
Support	Predictive API	Lower Rebooking Cost
Limit	No Maintenance Logs	Mixes Tech/Weather Bias
Future	Real-time ATC Feeds	Tactical Precision

Exhibit 9. Data Dictionary

Attribute	Type	Source	Description
ARR_DELAY	Double	Flight Data	Label: Total arrival delay (minutes)
IS_CANCEL	Boolean	Flight Data	Label: Binary flight status (Target)
WIND_GST	Double	Weather	Peak wind speed in knots at Origin
PRCP_SUM	Double	Weather	Cumulative 24h precipitation (mm)
SNOW_DP	Double	Weather	Snow depth on runway (cm)
FL_DENSITY	Integer	Calculated	Total hourly flights at specific Hub

Exhibit 10. Sample Transformation Query

```
SELECT
    f.FL_DATE,
    f.ORIGIN,
    f.DEP_DELAY,
    w.wind_gust_max,
```

```
    w.precip_sum,
    CASE WHEN f.CANCELLED = 1 THEN 'High Risk' ELSE 'Operational' END AS
Status_Flag
FROM flight_data f
JOIN weather_data w ON f.FL_DATE = w.date AND f.ORIGIN = w.station_id
WHERE w.wind_gust_max > 25 OR w.precip_sum > 10
ORDER BY f.DEP_DELAY DESC
LIMIT 100;
```